



First Steps Toward Automated AI Research

Early results from Recursive's automated AI research system on model training and GPU kernel benchmarks

JUNE 11, 2026

Today we are releasing early results from Recursive's automated AI research system. Across three benchmarks, the system achieves state-of-the-art results: in fixed-budget language model training, small-model training speed, and GPU kernel optimization.

The system automates the research loop for a target objective: it proposes an idea, implements it, runs an experiment, validates the result, and uses what it learns to choose the next experiment. It runs many research threads over long horizons, keeps useful context from prior experiments, combines promising branches, and puts results through validation for reward hacks and variance

before treating improved performance as real progress. It is designed to scale and harnesses principles of open-ended algorithms, building on ideas from previous work by our team and others into recursively self-improving AI.

We tested the system on benchmarks chosen for both practical importance and tight feedback loops. They stress three core levers of AI progress: better training algorithms, faster training, and more efficient use of hardware. They are also well suited to automated research because they have clear metrics, relatively low variance, and evaluators that can be hardened against reward hacks.

We are **open-sourcing artifacts** from these runs so others can inspect and build on the system's outputs.

Benchmark	Task Type	Metric	Previous State of the Art	Recursive	Improvement
NanoChat Autoresearch	Train a small language model to highest performance given a small compute budget	Validation BPB	0.9372	0.9109	0.0263 loss reduction, or a speedup to reach the same loss
NanoGPT Speedrun	Train a small language model to a certain performance as fast as possible	Training time required to reach a 3.28 validation loss	79.7 s	77.5 s	2.2s faster training
SOL-ExecBench	Optimize GPU kernels toward	Mean SOL score	0.699	0.754	18% reduction in gap to the

hardware	across	optimal
limits	235	performance
	kernels	estimate
		1.0

Case study 1: NanoChat Autoresearch

Andrej Karpathy's NanoChat [autoresearch repo](#) is a popular starting point for automated research systems. The task is to train a small language model to the lowest validation loss, measured in bits per byte (BPB), within a fixed five-minute budget on a single GPU. It is a natural test of our system because experiments are fast, variance is low, and reward hacks are relatively easy to detect.

Perhaps for those reasons, a public collaborative effort has already formed around this setup. [autoresearch@home](#) extends the original setup into a collaborative setting where several dozens of humans and hundreds of their agents collectively improve performance. That gives us a stronger comparison point than Karpathy's single overnight run. We wanted to test if our system could improve on solutions produced by an entire community of humans and agents.

Our system starts from the same initial seed solution the Autoresearch code starts from. We initially searched on NVIDIA H100 GPUs, then transferred the discovered solution to run on an NVIDIA B200 GPU for a fair comparison to public results. After removing minor reward hacks from the previous best autoresearch@home solution and evaluating it on 10 random seeds, its mean performance is 0.9372 BPB. Our system found a solution that reached 0.9109 BPB, a 0.0263 BPB improvement. Measured another way, our solution reaches the quality of Karpathy's original overnight autoresearch BPB in roughly 1.3x less training time than the best autoresearch@home solution.

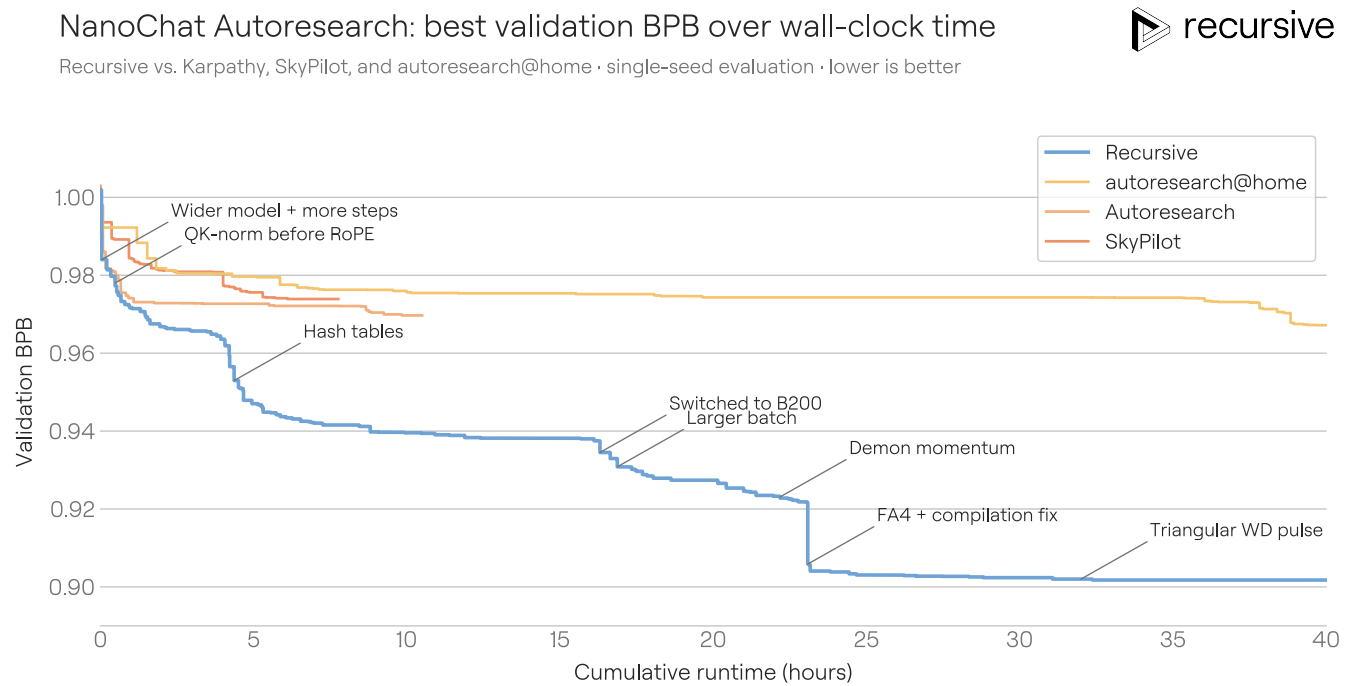


FIGURE 1

Autoresearch starts from an already optimized model with some non-trivial design decisions baked in. To this end, we tested whether our system could also make improvements from a much weaker starting point, a naive initial implementation (a vanilla Transformer with AdamW). Our system improved the model from 1.059 BPB to 0.9344 BPB (evaluated on an NVIDIA B200 GPU), again outperforming the best solution produced by the autoresearch@home community. This does not necessarily prove independent rediscovery, since the underlying models may know many public techniques including those used by or created by the autoresearch@home community, but it does show that the search process can assemble a competitive training stack from a much weaker starting point. The resulting solution also differed in several ways from the public best solution.

NanoChat Autoresearch: final validation BPB by solution

5 minutes of training on a single B200 · lower is better · mean $\pm$ standard error across seeds

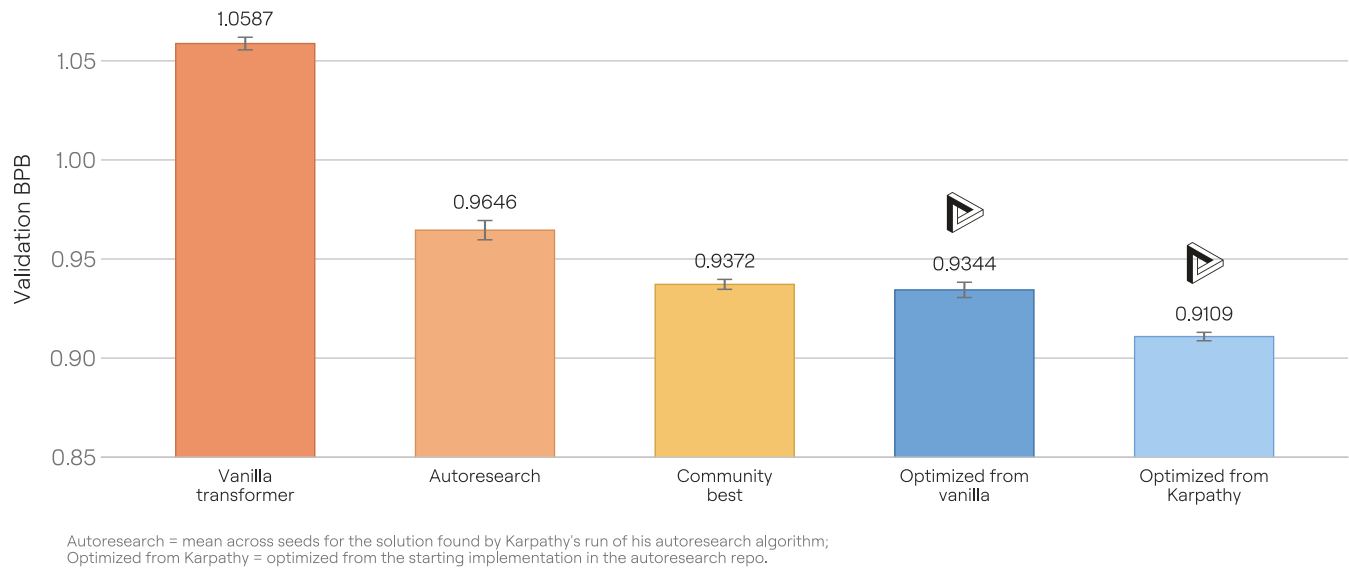


FIGURE 2

NanoChat Autoresearch: training loss over wall-clock time

Recursive vs. Karpathy seed and community best · single B200
band = min-max over 10 random seeds

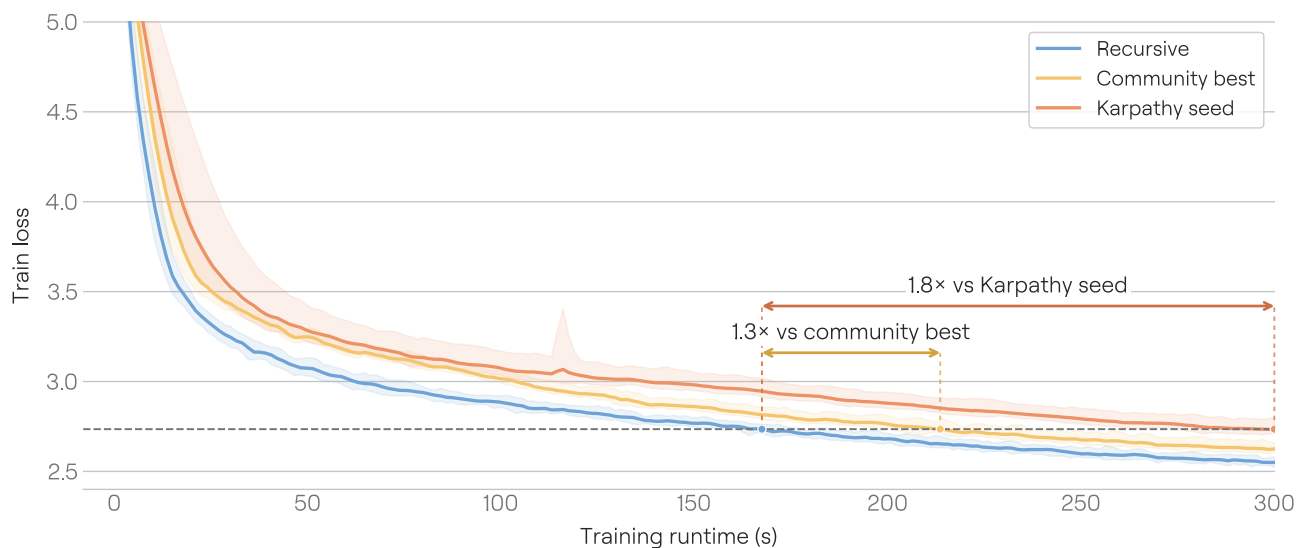


FIGURE 3

What modifications did our system come up with? The best solutions were

not driven by one trick. They combined changes to architecture, short-context memory, auxiliary losses, attention, optimizer behavior, weight decay schedules, compiler settings, and more.

One of the biggest gains came from a richer short-context memory mechanism. The baseline already uses value embeddings; our system extended this idea with hashed bigram and trigram embedding tables, mixed into the attention value path through learned gates. This gave the model a cheap way to use local n-gram information without paying the time cost of slower convolutional or attention-heavy alternatives.

This connects to recent work such as **DeepSeek Engram**, which explores hash tables as a sparsity axis. In our setting, hash tables can add 1-2 billion sparse parameters to a roughly 50M parameter model: most entries are inactive on any given batch, and lookup is cheap. Similar hash-table and n-gram ideas also appear in top NanoGPT Speedrun submissions. The system adapted this family of ideas to the fixed-budget setting by injecting hashed bigram and trigram embeddings into attention value vectors across multiple layers, with different hashes per layer to reduce repeated collisions. We are not aware of prior work using this exact variant.

The expandable boxes below include selected technical details from the system's solutions. We manually inspected these outputs and used AI-assisted analysis to understand the techniques and screen for reward hacks. We may still have missed errors in kernel optimization where we are not specialists, but that is also part of the point: the ideas presented here came from the system, not from our prior expertise.

Hash tables

The run optimizing the vanilla Transformer used some of the same techniques as our best solution, including hash tables and squared-ReLU MLPs. But it also converged on a different (yet equally competitive) final stack, including token-shifting, weight averaging before eval, and byte-level feature embeddings. This suggests the system was not merely repeating the same discoveries it found in the other run. The expandable box below shows a few modifications unique to the vanilla Transformer run.

Optimizing a vanilla Transformer

NanoChat shows how asking our system to improve fixed-budget training led to the discovery of many compounding, budget-aware improvements. The next test was whether the same process could still find gains after years of public human optimization on a benchmark. We tested that on NanoGPT Speedrun, whose best public solution has been highly optimized by the community over two years.

Case study 2: NanoGPT Speedrun

NanoGPT Speedrun is a similar task, yet it's much harder to beat the state of the art because a large community has been optimizing solutions for it for over two years. Instead of asking how low of a validation loss can be achieved in a fixed time budget, the benchmark asks how quickly a small GPT-style model can be trained to a fixed validation loss of 3.28 on the FineWeb text dataset, using a single HGX H100 8-GPU node.

This is a mature community effort, with 83 human record-setting contributions to the leaderboard so far and hundreds of proposed PRs. Since mid-2024, the training time has been pushed from roughly 45 minutes down to 79.7

seconds through a long sequence of primarily hand-engineered submissions. Given that the current solution is so well optimized, there are few obvious improvements left.

Starting from the current leading solution, our system found a set of additional optimizations that reduced training time from 79.7 seconds to 77.5 seconds while still meeting the leaderboard's validation-loss significance requirement (mean validation loss ≤ 3.28 at $p < 0.01$).¹ This is a similar or larger improvement than recent human contributions.

The 77.5 s solution: FP8 attention, exploration noise, and cautious embeddings

NanoGPT Speedrun: training time to a 3.28 validation loss

Track 1 · GPT-2 Small on FineWeb, 8×H100 · progress since Oct 2025 · prior leaderboard record 79.7 s (#83)

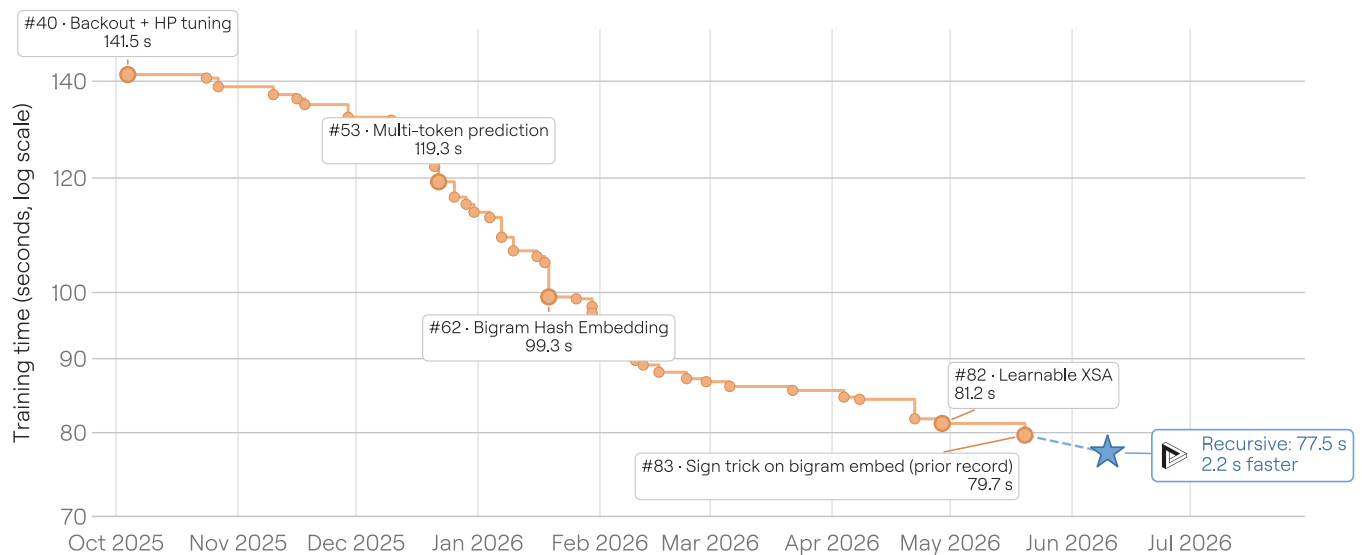


FIGURE 4

We also tested whether the system could make progress from a much weaker starting point. Starting from an earlier roughly 15-minute solution (we chose the

first working Python solution in the lineage of champions (entry #5), as earlier versions were in C), our system reached approximately 185 seconds in a few days, close to the human leaderboard's May 2025 roughly 180-second level. This should not be read as independent or unique discovery, since the underlying models may have seen the repository, but the system found a different final solution and added the overlapping contributions in a different order.

Analysis of the 185 s from-scratch solution: a different route to 3 minutes

The 77.5s solution was not a single optimization. It combined changes to attention precision, optimizer behavior, embedding updates, schedule choices, and fused GPU kernels. Each change had to save time without destabilizing training.

Despite an entire community of humans (sometimes with AI assistance) spending years working on this problem, Recursive's automated AI research system still discovered additional improvements. The next case study moves one level lower, from small-model training recipes to GPU kernels. Unlike the first two benchmarks, kernel optimization is closer to production systems work: it often determines the cost of real training and inference workloads.

Case study 3: SOL-ExecBench

The first two benchmarks optimize small language model training runs. **SOL-ExecBench** instead focuses on writing fast, correct GPU kernels: the small accelerator programs behind operations such as matrix multiplications, reductions, normalization layers, attention components, quantization routines, and fused blocks.

The benchmark contains 235 kernel-writing tasks derived from real workloads. Each task provides a simple reference PyTorch implementation that defines the signature, tensor shapes, data types, and numerical contract (what output the kernel must produce, and how close it must be to the reference implementation). The goal is to produce the same result within tolerance while running as fast as possible on NVIDIA Blackwell B200 GPUs.

The benchmark reports a Speed-of-Light (SOL) SOL-ExecBench score: 0.5 corresponds to the benchmark's optimized PyTorch baseline, and a score of 1.0 corresponds to the benchmark's analytical optimal performance estimate.

We ran our system across all 235 kernels jointly, so it could reuse its discoveries for better ways to do things across related tasks (e.g. patterns for memory movement, tiling, reductions, vectorization, and fusion). We provided standard profiling tools but did not specifically tune the system for kernel engineering. Aside from adding profiling tools, we use the same system to optimize kernels as in the other two benchmarks.

Our system achieved a mean NVIDIA SOL-ExecBench score of 0.754, an 18% reduction in the gap to the hardware limit from the previous leaderboard best of 0.699.

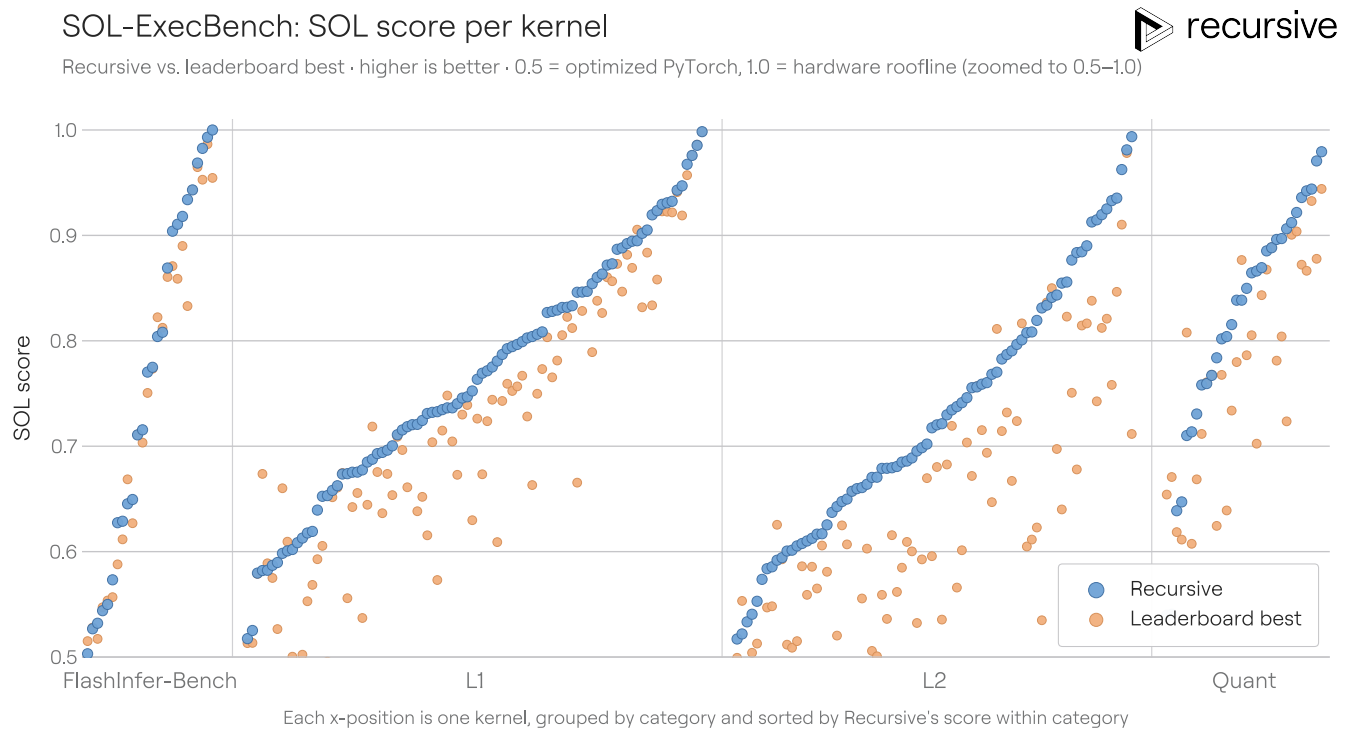


FIGURE 5

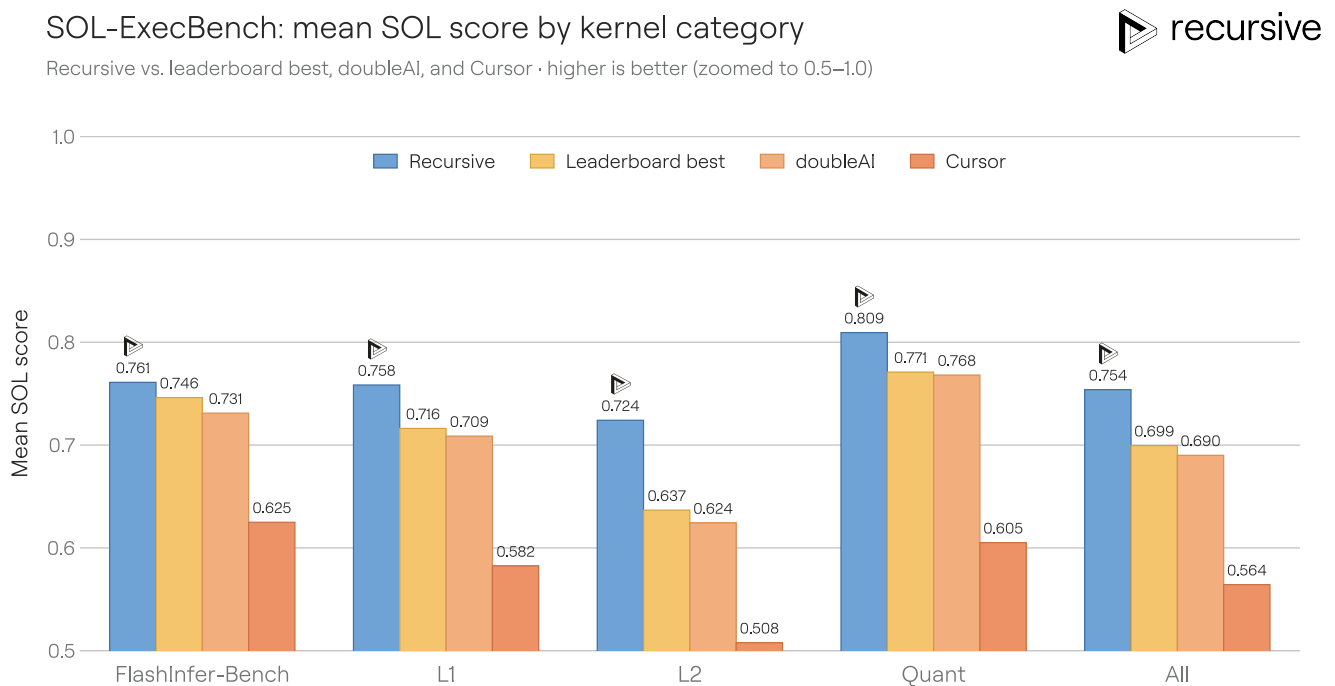


FIGURE 6

We checked a few high-performing kernels and found that the solutions include

a range of good kernel engineering practices and creative solutions. The expandable boxes provide a few examples for those interested in the details.

Example L1 kernel

045_fused_linear_gelu_grn_linear (L1)

Example L2 kernel

049_group_limited_topk_routing (L2)

Example Quant kernel

032_nvfp4_moe_expert_linear_with_gating (Quant)

Example FlashInfer-Bench kernel

012_gqa_paged_decode_h32_kv4_d128_ps1 (FlashInfer-Bench)

While reward hacking was an issue we contended with for all three benchmarks, it was particularly challenging on SOL-ExecBench. Some candidates exploited the evaluation setup instead of implementing genuinely faster kernels: caching outputs, relying on persistent state, or taking advantage of timing-harness details.

For that reason, we treated a correctness audit as part of the research system on all of the benchmarks. Promising improvements were passed through increasingly strict automated checks designed to distinguish genuine kernel improvements from benchmark-specific exploits. This substantially reduced reward hacks and became an important part of the loop itself: as the search became stronger, the evaluator had to become stronger too.

SOL-ExecBench demonstrates the ability of our system to improve an entirely

different part of the AI stack. It had to reason about low-level implementation choices, generate candidate kernels, run correctness and performance checks, and transfer useful patterns across related tasks.

What's Next

These results are an early sign that our system can push the frontier on AI training and infrastructure tasks, especially when the goal is well-defined, measurable, and quick enough to evaluate many times. The system made progress by compounding many discoveries: inventing new optimizations, recasting known ideas under tighter constraints, tuning implementation details that mattered, and composing improvements across modeling, optimization, and systems layers.

Throughout this work, and especially as our search becomes more powerful, a key challenge is reward hacking (i.e. making sure the system solves the intended task instead of exploiting loopholes that meet the letter of the task and score highly, but subvert the intention of the task). We implemented many techniques to avoid and detect such reward hacking, including iteratively improving a reward hacking detector with AI-assisted and/or human feedback. We expect this will remain necessary as we tackle ever more challenging real-world applications and create more powerful automated AI research algorithms. Aligning such systems to solve the spirit of the task, and not its letter, will be a grand challenge of creating systems that automate knowledge discovery and recursively self-improve in a way that is safe and helpful. We are excited to continue to work on that essential problem.

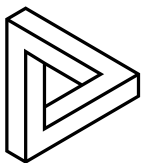
Many of the gains here improve efficiency. That matters because AI progress does not come only from larger models and more compute; it also comes

from making existing systems train faster, run cheaper, and use hardware more effectively. We expect systems like this to reduce the cost of intelligence: first by finding better engineering tradeoffs in today's systems, and over time by automating larger parts of the frontier research process itself.

We are **open-sourcing artifacts** from these runs so others can inspect and build on the system's outputs. If you're interested in building systems that make automated research more capable and beneficial for humanity, **please apply to join us.**

FOOTNOTES

1. We obtained our results on Modal HGX H100 8-GPU nodes and independently re-confirmed the numbers on Andromeda HGX H100 8-GPU nodes within noise. We are awaiting access to PrimeIntellect HGX H100 8-GPU nodes (the official hardware) to submit to the leaderboard.



© 2026 Recursive Superintelligence, Inc.
All rights reserved.

[Terms
of use](#)

[Privacy Policy](#) Follow
us

[LinkedIn](#) [X](#)